

SaaS Tiffin Delivery Platform

Architecture & Flow Diagrams

Prepared by	Sanchit Jha
Date	May 29, 2026
Version	1.0
Type	Technical Architecture

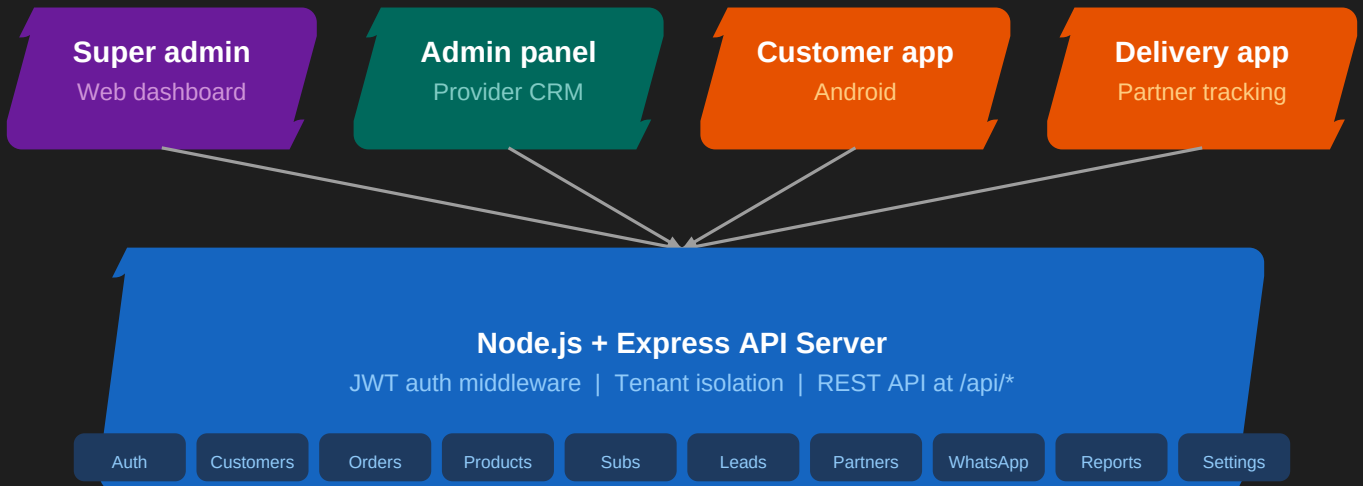
Diagrams

1. System Architecture Overview
2. Complete User Journey Flow
3. Order Lifecycle State Machine
4. WhatsApp Bot & App Distribution Flow
5. Multi-Tenant Isolation Architecture

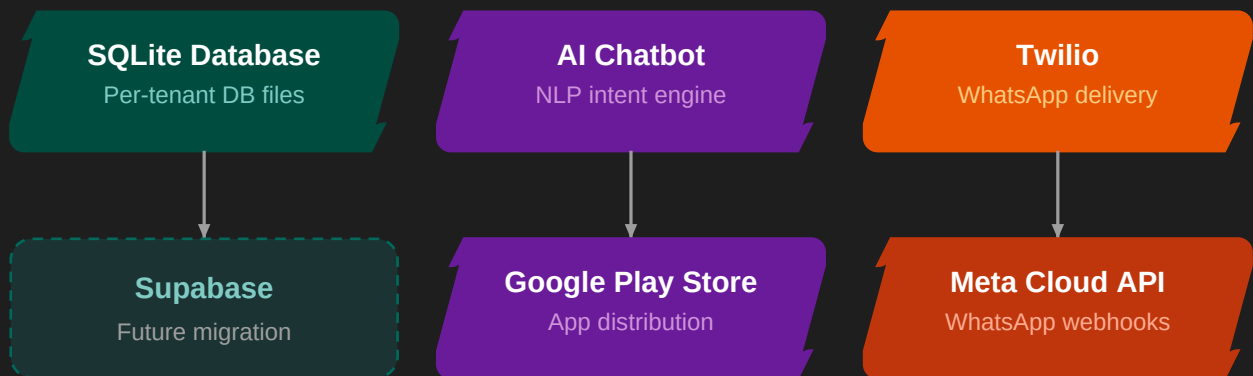
Diagram 1 — System Architecture Overview

How all components connect from user interfaces to databases

USER INTERFACES



DATA & SERVICES



Purple = platform infra Teal = data layer Orange = external services Dashed = planned

Diagram 2 — Complete User Journey Flow

How each role enters the platform and what they do

PHASE 1 — Onboarding

Super admin creates provider

Account, plan, WhatsApp number



PHASE 2 — Setup

Provider configures dashboard

Menu, areas, partners, settings



WhatsApp bot goes live

PHASE 3 — Discovery

Customer messages bot

WhatsApp / QR / referral

Customer downloads app

Provider-specific link

PHASE 4 — Ordering

Customer places order

Via app or WhatsApp bot



Provider confirms + cooks

Dashboard updates, kitchen starts



PHASE 5 — Delivery

Delivery partner dispatched

Pickup, transit, deliver



Order delivered

Stock updated, revenue logged



Super admin



Provider



Customer



Delivery partner

Diagram 3 — Order Lifecycle State Machine

Every state an order passes through, with who triggers each transition



Diagram 4 — WhatsApp Bot Message Routing

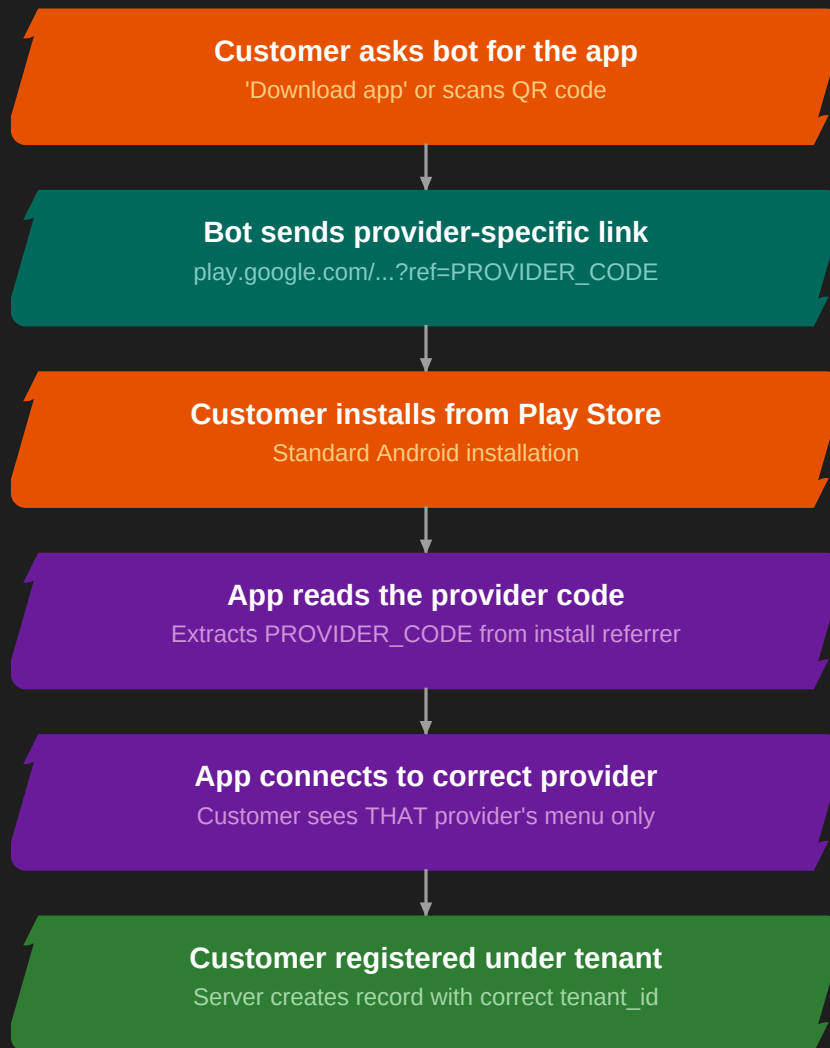
How incoming messages are routed to the correct provider

WHATSAPP MESSAGE ROUTING



Diagram 4b — App Distribution via WhatsApp

How customers download the app with automatic provider linking



ALTERNATIVE DISTRIBUTION

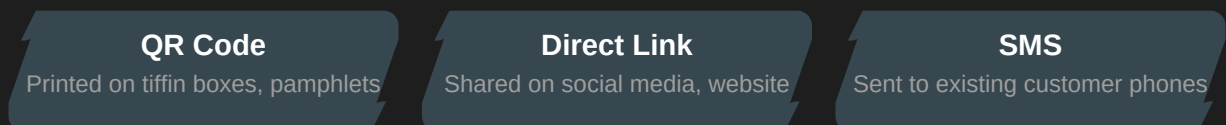


Diagram 5 — Multi-Tenant Isolation Architecture

4 layers of isolation preventing data leakage between providers



Diagram 5b — Conflict Prevention Matrix

How we handle every possible cross-tenant conflict scenario

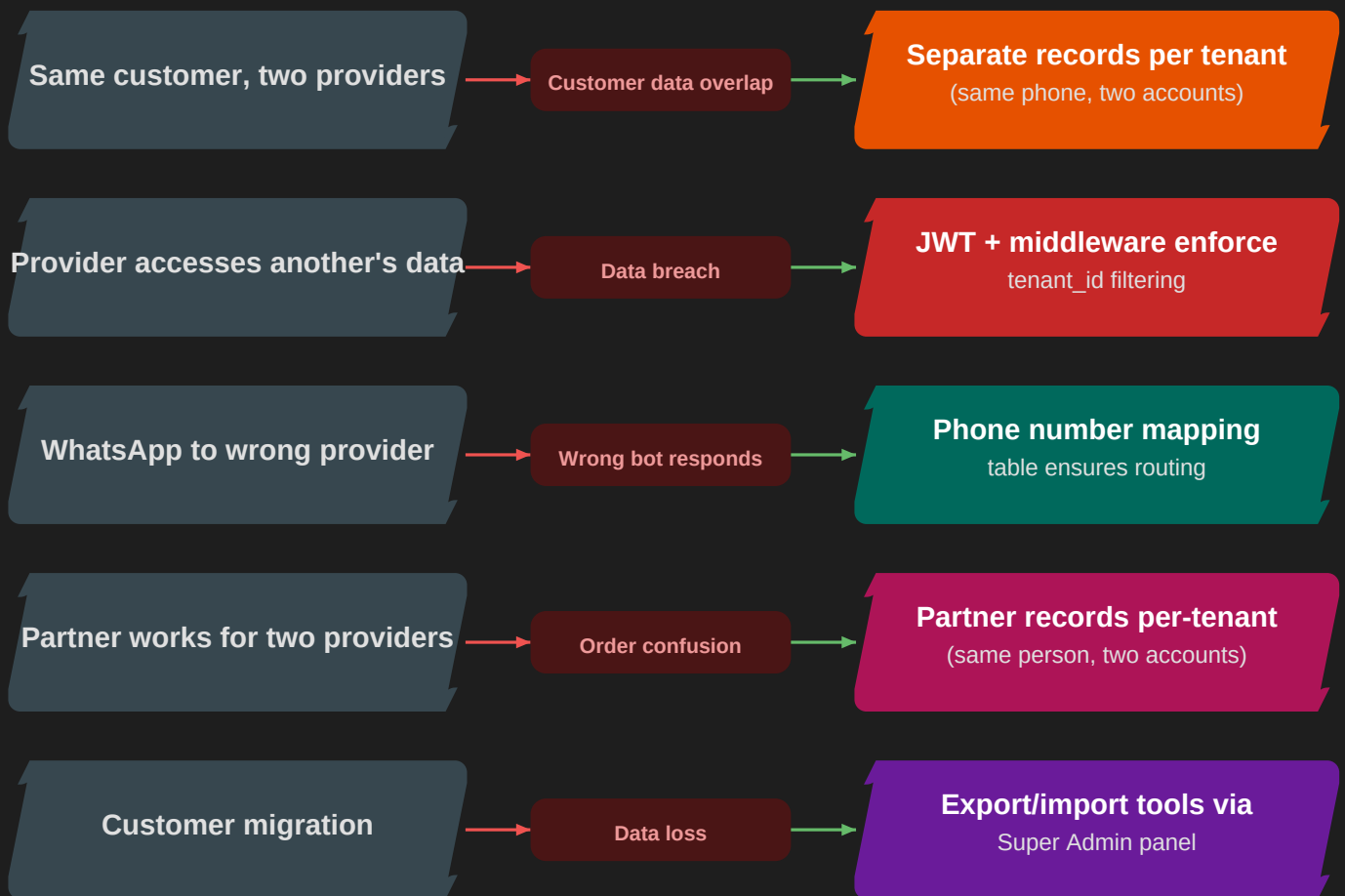


Diagram 6 — End-to-End Data Flow Matrix

What each role does at every stage of the platform lifecycle

Stage	Super Admin	Admin (Provider)	End User	Delivery Partner	WhatsApp Bot
Onboard	Creates account	Gets credentials	-	-	-
Setup	Registers WA number	Adds menu & partners	-	Added by provider	-
Go Live	Monitors metrics	Shares QR codes	-	-	Activated
Discovery	-	-	Messaging WA / QR	-	Welcome message
App DL	-	-	Downloads app	-	Sends link
Order	-	Receives order	Places order	-	Creates order
Cook	-	Confirms & cooks	Gets confirm	-	Status update
Dispatch	-	Assigns partner	Sees OFD status	Picks up order	-
Deliver	-	Dashboard updated	Receives tiffin	Marks complete	Delivery confirm
Growth	Converts trials	Uses reports	Subscribes daily	Covers areas	Handles repeats

Super Admin

Provider

Customer

Partner

Bot

Diagram 7 — Use Case Diagram

All actors and their interactions with the system

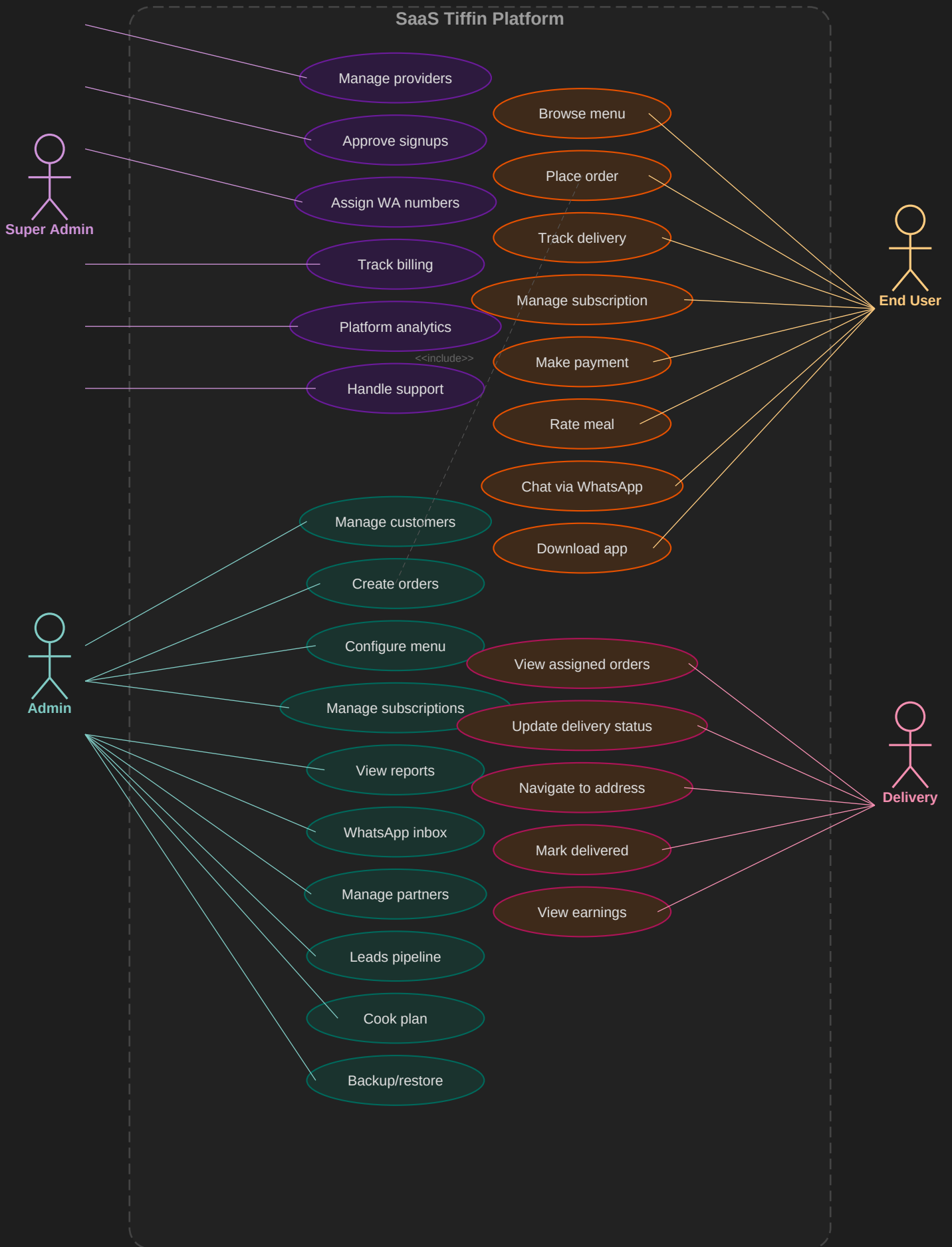


Diagram 8 — Services / Module Architecture

Layered service architecture showing all modules and their dependencies

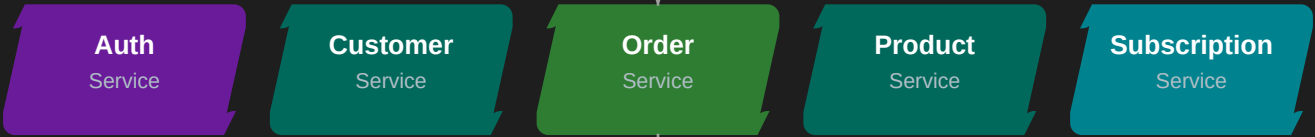
PRESENTATION LAYER



API GATEWAY



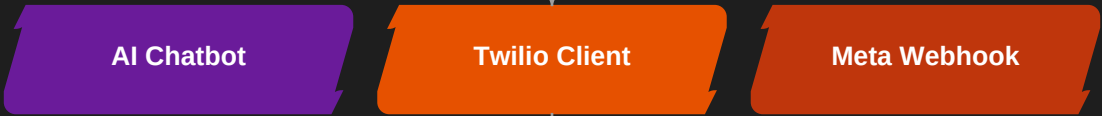
BUSINESS LOGIC LAYER



BUSINESS LOGIC LAYER (cont.)



INTEGRATION LAYER



DATA LAYER

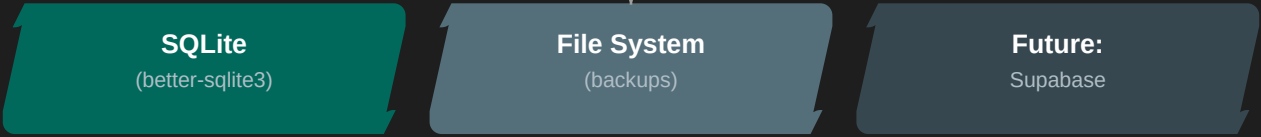


Diagram 9 — Class Diagram

Core data models and their relationships



Diagram 10 — Entity Relationship (ER) Diagram

Database entities and their relationships (from dabbabox.db)



leads, whatsapp_messages, activity, settings — standalone tables (no foreign keys)

Diagram 11 — Activity Diagram (Order Flow)

Complete activity flow from order placement to delivery

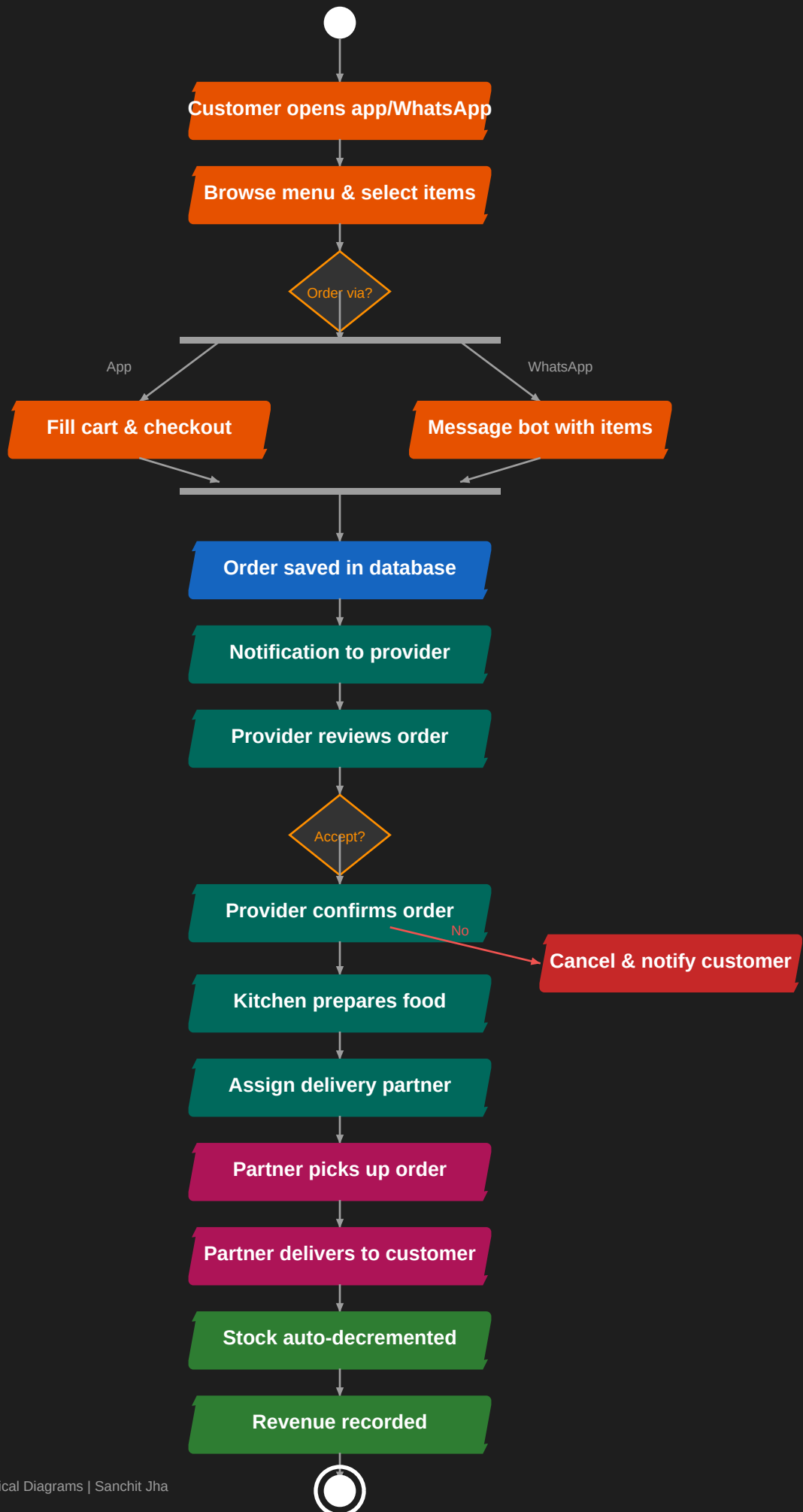
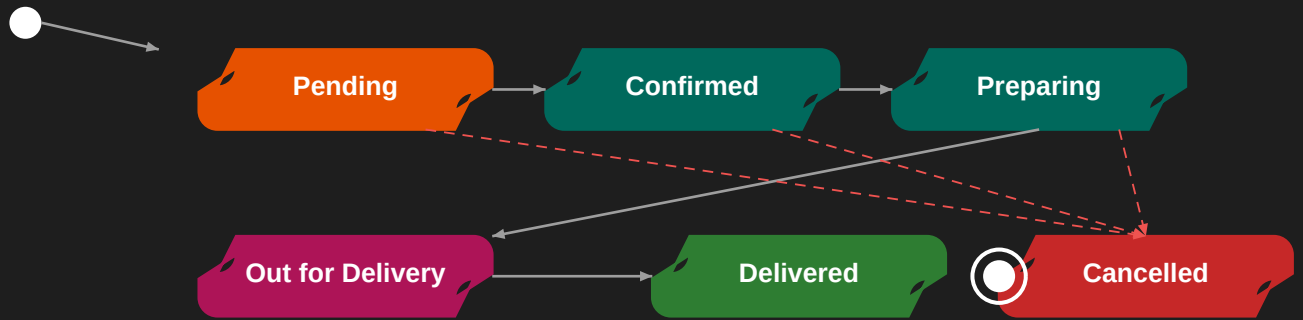


Diagram 12 — State Diagrams

State transitions for Orders, Subscriptions, and Leads

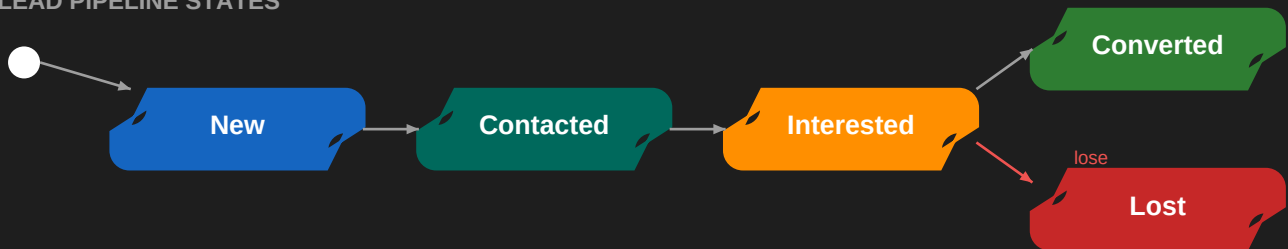
ORDER STATES



SUBSCRIPTION STATES



LEAD PIPELINE STATES



STATUS ENUMERATIONS

Order Status	pending	confirmed	preparing	out_for_delivery	delivered	canceled
Sub Frequency	daily	weekly	biweekly	monthly		
Lead Stage	new	contacted	interested	converted	lost	
WA Direction	in	out				
User Role	admin	super_admin				

Diagram 13 — Database Tables (Page 1/2)

Complete schema from dabbabox.db — all tables with column types

users

PK id	TEXT
email	TEXT UNIQUE NOT NULL
name	TEXT NOT NULL
password	TEXT NOT NULL (bcrypt)
role	TEXT DEFAULT 'admin'
created_at	INTEGER NOT NULL

customers

PK id	TEXT
name	TEXT NOT NULL
phone	TEXT
email	TEXT
area	TEXT
address	TEXT
tags	TEXT (JSON array)
notes	TEXT
preferences	TEXT (JSON obj)
created_at	INTEGER NOT NULL

products

PK id	TEXT
name	TEXT NOT NULL
emoji	TEXT
category	TEXT
price	REAL DEFAULT 0
stock	INTEGER DEFAULT 0
low_stock_at	INTEGER DEFAULT 5
description	TEXT

orders

PK id	TEXT
customer_id	TEXT FK -> customers
partner_id	TEXT FK -> partners
delivery_address	TEXT
items	TEXT (JSON array)
customizations	TEXT (JSON array)
status	TEXT NOT NULL
notes	TEXT
delivery_fee	REAL DEFAULT 0
subtotal	REAL DEFAULT 0
total	REAL DEFAULT 0
events	TEXT DEFAULT '[]'
created_at	INTEGER NOT NULL

INDEXES:

```
customers(name), customers(phone)
orders(customer_id), orders(status), orders(created_at)
```


Diagram 13 — Database Tables (Page 2/2)

subscriptions

PK id	TEXT
name	TEXT NOT NULL
customer_id	TEXT FK -> customers
frequency	TEXT NOT NULL
status	TEXT NOT NULL
next_delivery	TEXT
items	TEXT DEFAULT '[]'
created_at	INTEGER NOT NULL

partners

PK id	TEXT
name	TEXT NOT NULL
phone	TEXT
area	TEXT
vehicle	TEXT
active	INTEGER DEFAULT 1

leads

PK id	TEXT
name	TEXT NOT NULL
phone	TEXT
source	TEXT
stage	TEXT DEFAULT 'new'
follow_up	TEXT
notes	TEXT
created_at	INTEGER NOT NULL

whatsapp_messages

PK id	TEXT
phone	TEXT NOT NULL
name	TEXT
direction	TEXT NOT NULL
text	TEXT NOT NULL
unread	INTEGER DEFAULT 1
at	INTEGER NOT NULL

stock_moves

PK id	TEXT
product_id	TEXT FK -> products
delta	INTEGER NOT NULL
reason	TEXT
at	INTEGER NOT NULL

activity

PK id	TEXT
at	INTEGER NOT NULL
text	TEXT NOT NULL
kind	TEXT
ref	TEXT

settings

PK key	TEXT
value	TEXT NOT NULL

INDEXES:

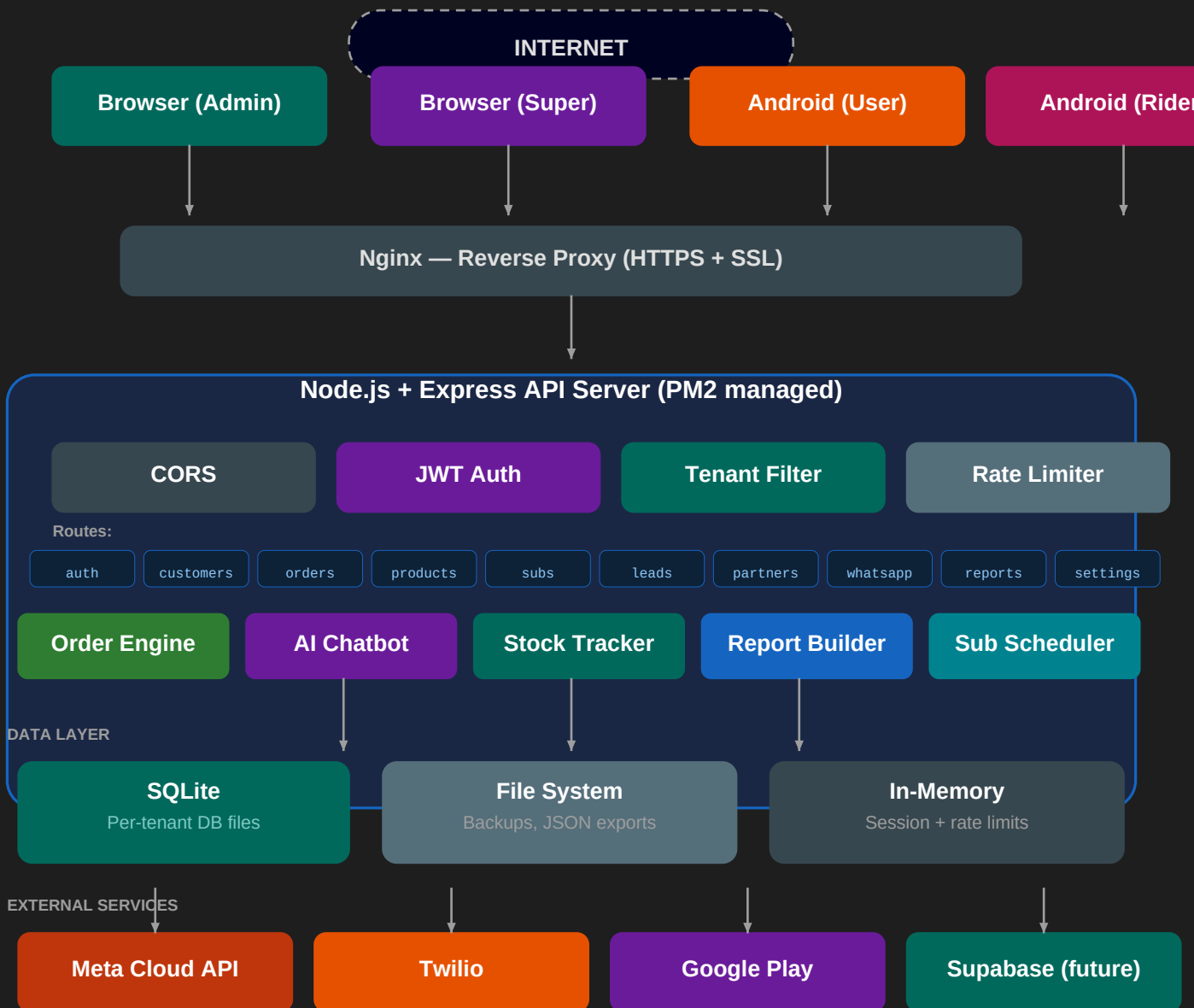
```
whatsapp_messages(phone), whatsapp_messages(at)
activity(at)
```

FOREIGN KEYS:

```
orders.customer_id -> customers.id (CASCADE)
orders.partner_id -> partners.id (SET NULL)
subscriptions.customer_id -> customers.id (CASCADE)
stock_moves.product_id -> products.id (CASCADE)
```

Diagram 14 — System Design (High-Level)

How anyone interacts with the product — complete working system



MONTHLY COST (100 providers):

VPS (4GB): Rs 800-1500 Domain+SSL: Rs 100/yr WhatsApp: Rs 0.12-0.86/msg Twilio: \$0.005/msg Total fixed: ~Rs 2,000/mo

Diagram 15 — Sequence Diagram (Place Order)

Step-by-step interaction when a customer places an order

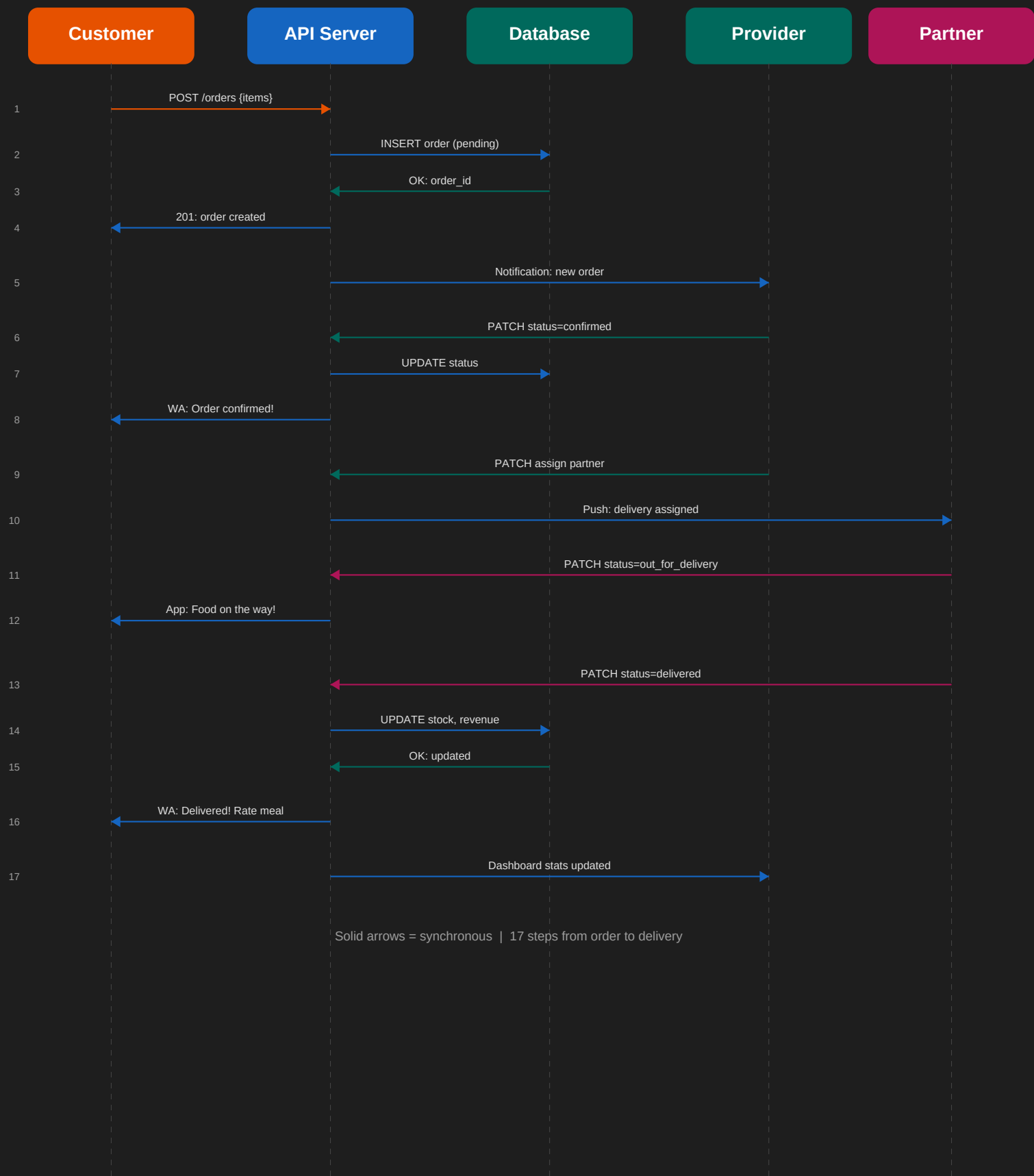


Diagram 16 — Product Interaction Overview

How each user type interacts with the platform — complete action list



Diagram 17 — Deployment & Infrastructure

Production deployment with all infrastructure components

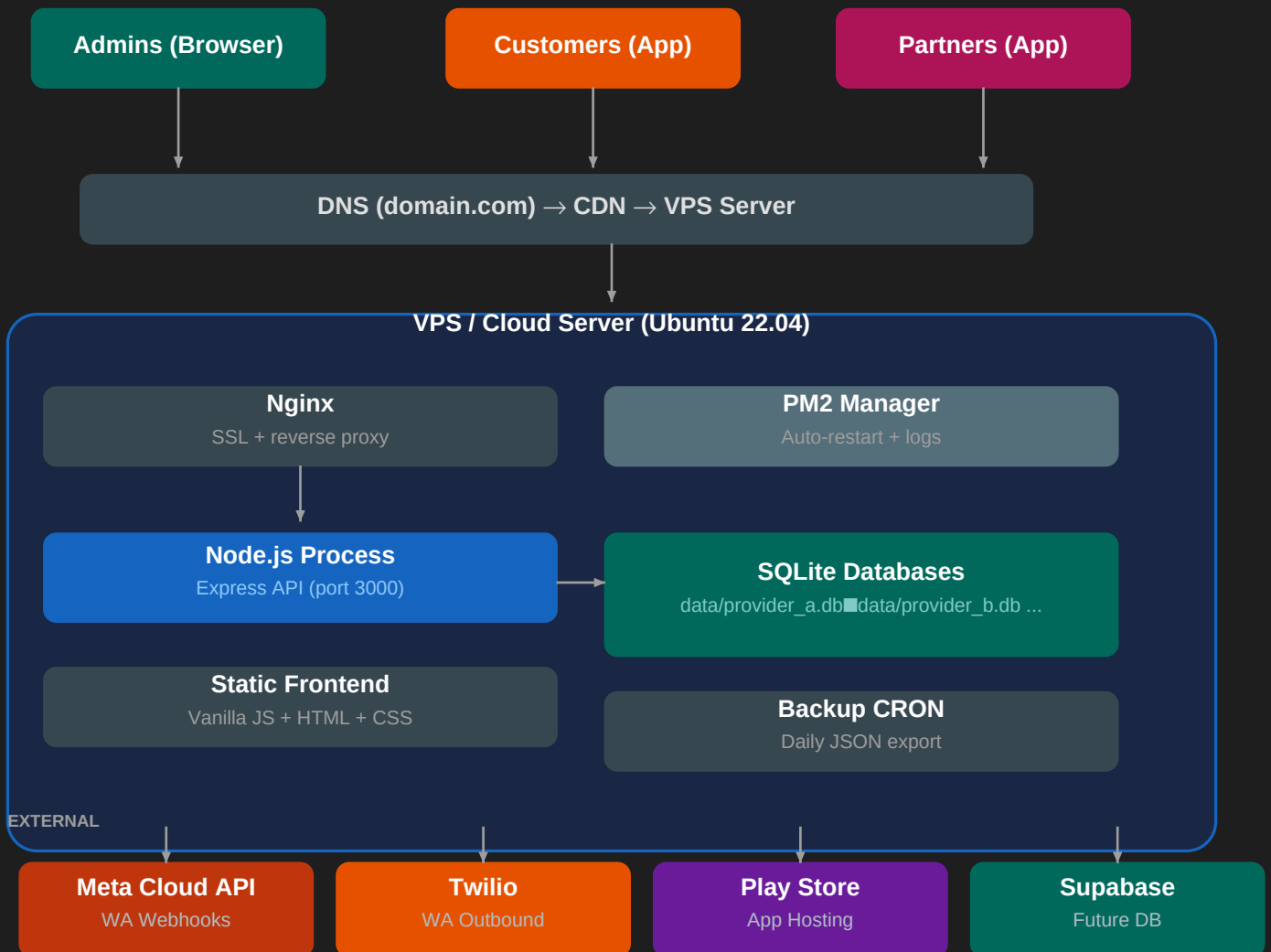


Diagram 18 — Function Map (All API Functions)

Every function in the system grouped by module

Auth	<pre>POST /login → validate → generateJWT() GET /me → verifyToken → getUserById() POST /change-password → hash → update()</pre>
Customers	<pre>GET / → list(search, area) POST / → create(name, phone, email, area) GET /:id → getWithOrders → calcLTV() PUT /:id → update(fields) DELETE /:id → delete [CASCADE]</pre>
Orders	<pre>GET / → list(status, date) POST / → create(custId, items, addr) PATCH /:id/status → update → logEvent → adjustStock PATCH /:id/partner → assignPartner(id) PUT /:id → update(fields)</pre>
Products	<pre>GET / → list(category) POST / → create(name, price, stock) PUT /:id → update(fields) POST /:id/stock → adjust(delta) → logMove()</pre>
Subscriptions	<pre>GET / → list(status) POST / → create(custId, freq, items) PATCH /:id/toggle → pauseOrResume() DELETE /:id → delete()</pre>
Leads	<pre>GET / → list(stage) POST / → create(name, phone, source) POST /:id/convert → toCustomer() PUT /:id → update(stage, followUp)</pre>
WhatsApp	<pre>POST /webhook → parse → botReply → sendTwilio GET /threads → conversations → matchCustomer POST /reply → save → sendViaTwilio() POST /mark-read → markAllRead(phone) GET /unread-count → countUnread()</pre>
Reports	<pre>GET /summary → revenue, AOV, topCustomers GET /dashboard → trend, statusCounts, popular GET /cook-plan → aggregateItems, customizations GET /area-intelligence → areaRevenue, trend GET /customizations → customizationStats()</pre>
Settings	<pre>GET / → getAllSettings() PUT /:key → upsert(key, value) GET /backup/download → dumpAllTables → JSON POST /backup/restore → wipe → reimport POST /reset → wipe → seed()</pre>

HELPER FUNCTIONS (db.js):

```
uid(prefix) → generate unique ID
json.parse(s, fallback) → safe JSON parse
json.stringify(v) → JSON.stringify wrapper
rowToCustomer(r) → map DB row to Customer obj
rowToOrder(r) → map DB row to Order obj
rowToProduct(r) → map DB row to Product obj
rowToSubscription(r) → map row to Sub obj
rowToLead(r) → map row to Lead obj
rowToWhatsApp(r) → map row to WhatsApp msg obj
rowToPartner(r) → map row to Partner obj
```